

KenSupportTracker Reflective Report

Kenneth Kenneth 24956195

<https://github.com/kenkie123/KenITSupportTracker>

1. Domain Understanding

Developing KenSupportTracker changed the way I understood the purpose of a support ticket, because I initially viewed it mainly as a way to record that an issue had occurred, whereas through the development process I realised that a useful ticket also needs to preserve enough context for another person to understand the problem, take responsibility for investigating it, and later explain how it was resolved. My previous experience working with service-centre systems influenced this part of the design, particularly because hardware and software issues are often more specific than they first appear. For example, a kiosk may still respond when a customer touches the screen but fail to print a ticket, which means describing the issue only as “kiosk broken” would not give a technician enough information to identify whether the problem relates to the printer, display, software or another component. This influenced my decision to require an issue description and location alongside the title and requester, as unclear reporting in a service-centre environment can delay investigation and extend the disruption experienced by both staff and customers. I also learnt that assigning a ticket and resolving it should be treated as separate stages rather than as one action, because recording a technician’s name establishes who is responsible for investigating the issue but does not prove that the fault has actually been repaired. This distinction led to the use of Open, Pending and Resolved statuses within the application, although I also recognised that the term “Pending” may not always communicate the same meaning in every support environment, as it can also refer to waiting for more information or waiting for a customer response. For that reason, if the application was developed beyond the MVP stage, I would validate this terminology with active IT technicians rather than assume that the current labels are appropriate in all cases.

2. Architecture decisions

I chose three use cases because submitting, assigning and resolving tickets are distinct business decisions. `SubmitSupportTicketUseCase` protects the completeness of a new report. `AssignSupportTicketUseCase` establishes responsibility. `ResolveSupportTicketUseCase` protects the conditions for closure. Separating these operations from `SwiftUI` allows their rules to remain consistent when the interface changes. Submission trims whitespace and requires a title, description, requester and location. The title limit of 100 characters is a design choice that keeps the ticket list readable, while longer explanations belong in the description. Assignment requires a technician name and rejects resolved tickets. Resolution requires pending status, an assigned technician and non-empty notes. Blocking changes to resolved tickets helps preserve the recorded outcome, although the current application does not provide a full audit history. `SupportTicket` is a struct because it represents a record as a value. `TicketViewModel` is a class because multiple screens observe the same changing ticket list. The `TicketRepository` protocol represents loading and saving the support-ticket register, while `LocalTicketRepository` implements this using JSON. This allows use cases to depend

on the required behaviour rather than file-handling details. The use of JSON was proportionate to a small, single-device MVP, but it introduces responsibility for format compatibility. Another deliberate decision was updating the published ticket list only after saving succeeds, or the interface could suggest that an assignment or resolution was complete even though it had not been stored.

3. Human-system design

A significant human error is closing a ticket before checking the repair. `ResolveTicketView` asks the technician to describe the fix and how it was checked, then requests confirmation. The use case separately validates assignment, status and notes. Confirmation therefore supports deliberate action, while validation prevents incomplete closure. Though this protection has limits, as non-empty notes do not prove that a repair was completed successfully, and the application cannot inspect the affected equipment. The technician remains responsible for verification. This distinction shaped the human-system boundary: the person investigates and judges the outcome; the application records the decision and enforces minimum requirements. If saving fails, the resolution form remains open with its notes and displays recovery guidance. Preserving this work reduces repeated effort. However, the Cancel button still dismisses the form without a discard warning, so the current design does not protect against every accidental loss. During development, existing tickets also stopped loading after a change to the ticket status structure. Reinstalling the app restored operation because the old stored data was removed, but I did not capture the original decoding error, so I cannot confirm the exact cause. This revealed a weakness in my unit tests, because they test business rules and simulated storage failures but do not test compatibility with older JSON files.

4. What I would implement next

With another two weeks, I would prioritise versioned storage and migration. Resetting disposable development data was practical, but asking a technician to delete real support history would undermine the application's purpose. This makes data continuity more valuable than adding another interface feature. I would also introduce a schema version and migration logic within the data layer. Recognised older status values would be converted deliberately, with the original file preserved until conversion and saving succeeded. Unknown or damaged formats would produce recovery guidance without silently replacing the register. I would test this using older-format fixtures, malformed files and actual save-and-reload checks. These tests would complement the existing use-case coverage by examining the storage boundary that failed during development. The experience taught me that solution engineering includes maintaining trust as software changes: a feature is only useful if the stakeholder can rely on both its current behaviour and the records it preserves.